Asclena AI: Comprehensive Technical Report

Theoretical Foundation & Implementation Architecture

Chapter 1

Philosophy and Core Concept

1.1 The Motivation: Beyond the Black Box

The deployment of Artificial Intelligence in healthcare faces a critical dilemma known as the "Black Box Problem." Traditional Deep Learning models, particularly Convolutional Neural Networks (CNNs), excel at pattern recognition but lack interpretability. Conversely, modern Generative AI models (LLMs/VLMs) offer excellent communication skills but suffer from "Hallucinations"—generating confident but factually incorrect medical advice.

Asclena addresses this by rejecting the idea of a single monolithic model. Instead, it adopts a *Epistemic Reasoning and Methodological Skepticism* (a consultation team), separating the cognitive processes into three distinct epistemic roles:

1. **Perception:** Quantitative analysis of visual data (CNNs).
2. **Interpretation:** Semantic understanding and reporting (VLMs).
3. **Verification:** Fact-checking against established medical literature (Agentic RAG).

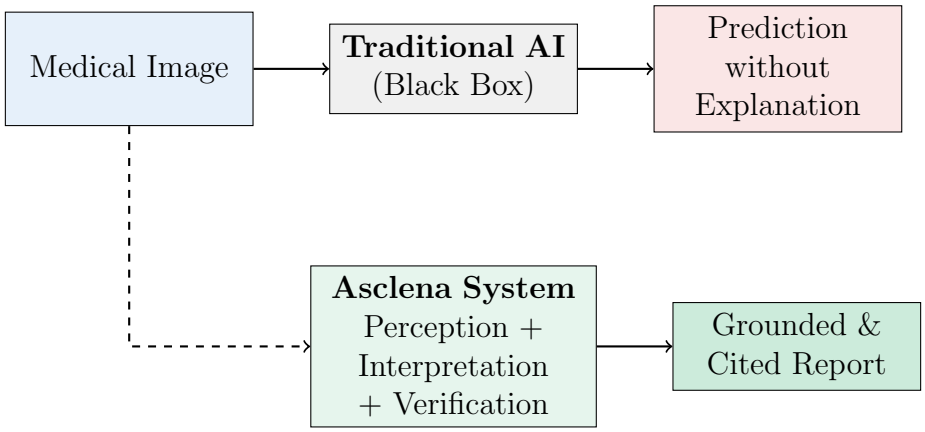

Figure 1.1: Conceptual Shift: From Black Box prediction to Asclena's transparent reasoning pipeline.

Chapter 2

Theoretical Foundation: Deep Learning in Medical Imaging

This chapter provides a comprehensive theoretical background on the application of Deep Learning in the medical domain. It explores the mathematical foundations of Convolutional Neural Networks (CNNs) and the evolution of architectures suitable for diagnostic imaging.

2.1 Convolutional Neural Networks (CNNs)

2.1.1 Core Concept

A Convolutional Neural Network (CNN) is a specialized neural architecture designed to process data with a known grid-like topology. In medical imaging, this data can be 2D (e.g., X-Rays, Histopathology slides) or 3D volumetric data (e.g., CT scans, MRI).

Unlike fully connected networks that treat inputs as a flat vector, CNNs operate on the premise of **"Local Connectivity"** and **"Parameter Sharing."**

Medical Importance (Translation Invariance): In medical diagnostics, a lesion (e.g., a tumor or nodule) can appear anywhere within an organ. The *translation invariance* property of CNNs ensures that the network recognizes the pathological feature regardless of its spatial location (e.g., whether the tumor is in the upper or lower lobe of the lung). This eliminates the need for manual alignment of images.

2.1.2 The Anatomy of a CNN

A standard CNN is composed of a sequence of layers that transform the input volume into an output class or map. Each layer plays a critical role in extracting clinically relevant information.

1. Convolutional Layer (The Feature Extractor)

The core building block. A set of learnable filters (kernels) slides over the input image.

- **Mathematical Operation:** It performs a dot product between the filter entries and the input, producing a "Feature Map."

- **Clinical Significance (Hierarchical Learning):**
  - *Early Layers:* Act as "Edge Detectors," identifying anatomical boundaries such as the rib cage, heart borders, or diaphragm.
  - *Deep Layers:* Capture abstract "Textural Features." In conditions like Pneumonia, the disease manifests not as a shape but as a "cloudy" texture (Opacities). The deep convolutional layers are capable of distinguishing these subtle texture changes from healthy tissue.

2. Activation Functions (Non-Linearity)

Linear operations alone cannot model complex biological data. Activation functions introduce non-linearity.

- **ReLU (Rectified Linear Unit):** $f(x) = max(0, x)$.
- **Medical Advantage:** Biological structures are highly non-linear and complex. ReLU allows the network to learn intricate decision boundaries between healthy and diseased tissues efficiently, accelerating the training process without suffering from the vanishing gradient problem in early stages.

3. Pooling Layers (Downsampling)

- **Function:** Reduces the spatial dimensions (Resolution) to decrease computational cost and control overfitting.
- **Types:** *Max Pooling* (takes the strongest signal) and *Average Pooling* (smooths the signal).
- **Medical Advantage:** *Max Pooling* acts as a feature selector. For example, if a small but high-intensity calcification (micro-calcification) exists in a mammogram, Max Pooling preserves this critical high-value signal while discarding less relevant background noise.

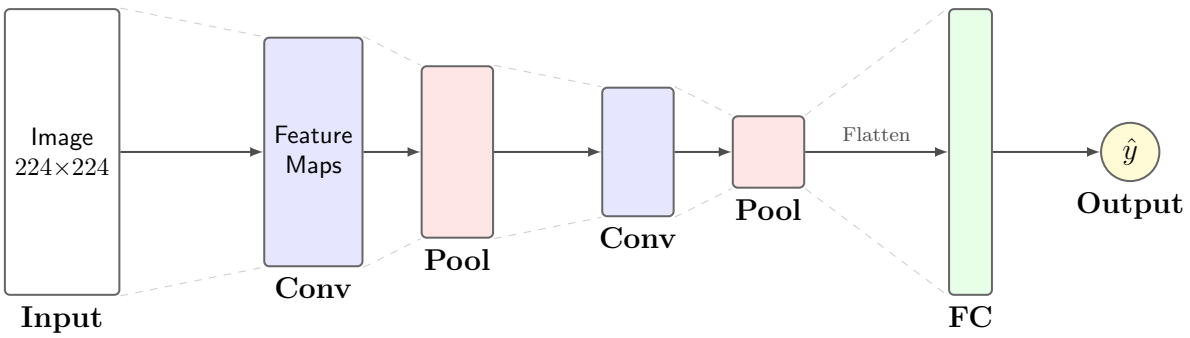

Figure 2.1: Schematic of the CNN Pipeline: The diagram highlights the progressive downsampling (represented by decreasing box height) to extract high-level features, followed by a dense classification layer.

2.2 Evolution of Architectures in Medicine

The choice of architecture dictates the model's ability to learn medical features. The field has evolved significantly to address the challenges of depth and feature preservation.

2.2.1 1. ResNet (Residual Networks)

As networks became deeper to capture more complex pathologies, training became impossible due to the "Vanishing Gradient" problem. ResNet introduced "Skip Connections" (or Identity Shortcuts).

- **Concept:** It allows the gradient to bypass layers, flowing directly backward during backpropagation.
- **Medical Use Case:** ResNet is the standard backbone for detecting complex diseases that require very deep semantic understanding, such as differentiating between various types of malignant skin lesions (Melanoma vs. Nevus).

2.2.2 2. DenseNet (Densely Connected Networks)

DenseNet took the skip connection idea further. Instead of adding inputs (like ResNet), it *concatenates* them.

- **Feature Reuse:** Every layer receives inputs from all previous layers.
- **Why it is Critical for X-Rays:** In medical imaging, "Low-level" features (edges/-textures) are often as important as "High-level" features.
- **Clinical Advantage:** DenseNet preserves fine-grained details throughout the network. This makes it superior for detecting pathologies like **Infiltration** or **Nodules**, where the disease manifests as a subtle local variation in tissue density rather than a gross shape change. The "collective knowledge" of all layers is available for the final diagnosis.

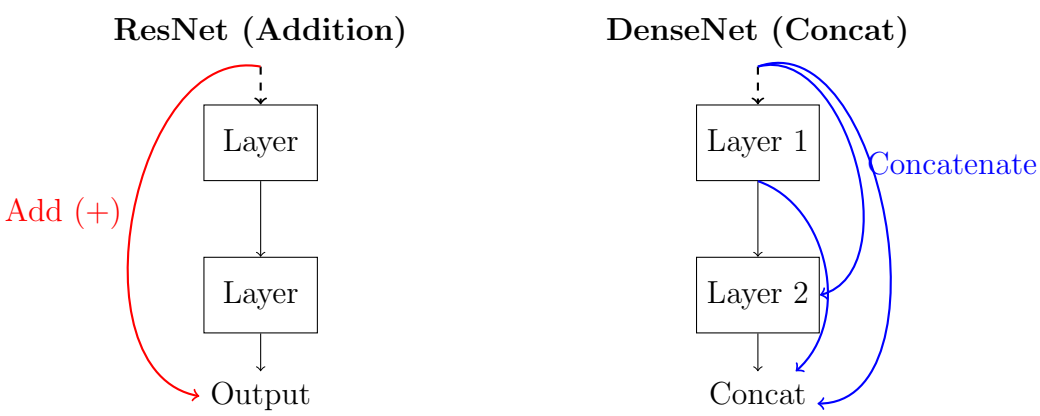

Figure 2.2: Architectural Comparison: ResNet (left) adds features, while DenseNet (right) concatenates them. Concatenation preserves the "original texture" of the image, which is vital for medical diagnosis.

2.2.3 3. EfficientNet

While previous models focused on scaling one dimension (usually depth), EfficientNet introduces "Compound Scaling."

The Concept of Compound Scaling

EfficientNet demonstrates that balancing network depth, width, and image resolution leads to better performance than scaling any single dimension alone.

$$\text{Depth: } d = \alpha^{\phi}, \quad \text{Width: } w = \beta^{\phi}, \quad \text{Resolution: } r = \gamma^{\phi}$$

Medical Advantage (Efficiency at Scale): In a real-world hospital environment, latency is critical. EfficientNet allows Asclena to process high-resolution DICOM images rapidly without requiring massive computational infrastructure. It achieves state-of-the-art accuracy with significantly fewer parameters (FLOPS), ensuring that the "Perception Layer" of the system remains responsive even under high load.

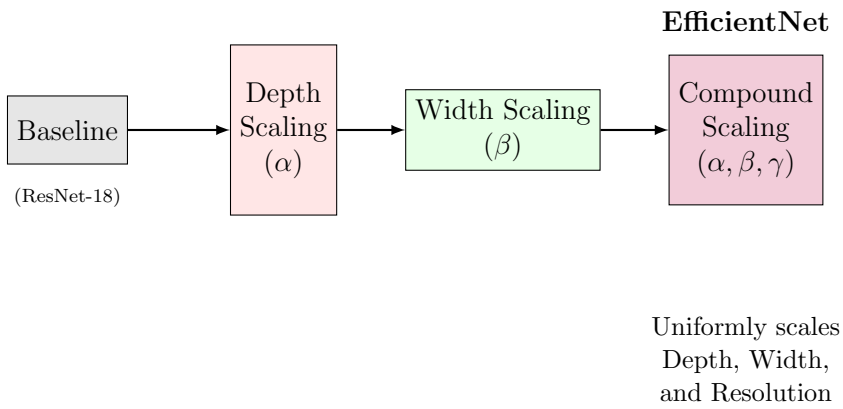

Figure 2.3: EfficientNet Strategy: Unlike traditional scaling (middle blocks), Compound Scaling (right) expands the network in all dimensions simultaneously to maximize clinical accuracy vs. computational cost.

2.3 Computer Vision Tasks in Medicine

Deep Learning is not just about classification. In the medical domain, we perform several distinct tasks, each solving a specific clinical problem:

2.3.1 1. Classification (Diagnosis)

Assigning a diagnostic label to an image.

- **Binary:** Normal vs. Abnormal.
- **Multi-Label:** Identifying multiple co-existing conditions (e.g., Cardiomegaly + Effusion), which is common in clinical practice.
- **Role:** Triage tool to prioritize sick patients in a work-list.

2.3.2 2. Segmentation (Delineation)

The process of partitioning an image into meaningful segments.

- **Role:** Critical for radiotherapy planning (outlining the tumor volume) and quantitative analysis (measuring organ volume).
- *Architecture:* **U-Net** is the standard for this task.

2.3.3 3. Detection (Localization)

Identifying the location of a lesion using a Bounding Box.

- **Role:** Acts as a "Second Reader" for radiologists, highlighting suspicious regions (Region of Interest - ROI) to ensure no subtle nodule is missed during visual inspection.

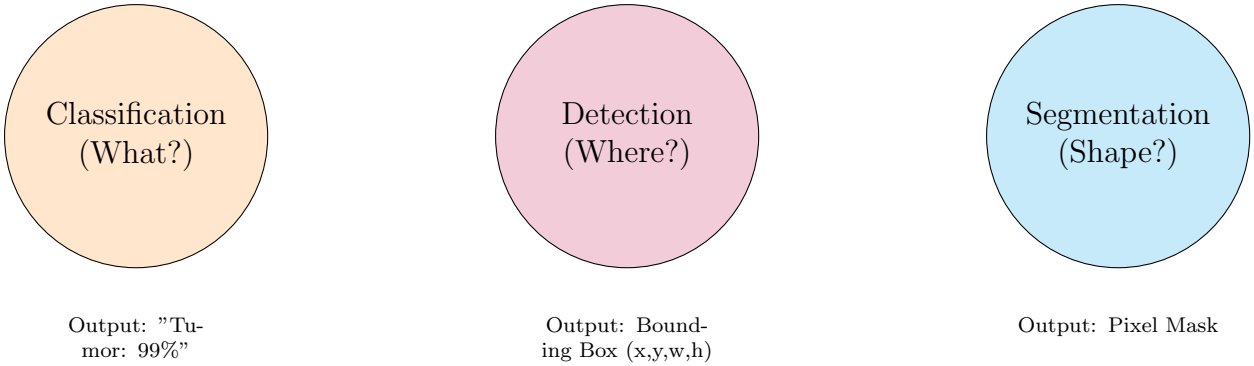

Figure 2.4: The three primary Computer Vision tasks in Medical Imaging: Classification gives the *diagnosis*, Detection gives the *location*, and Segmentation gives the *exact morphology*.

2.4 Vision-Language Models (VLMs) in Medical Imaging

2.4.1 Overview & Motivation

While Convolutional Neural Networks (CNNs) have revolutionized medical image analysis, they are traditionally limited to specific tasks like classification or segmentation. They answer "What is this?" but struggle to answer "Why?. **Vision-Language Models (VLMs)** bridge this gap by combining Computer Vision (CV) and Natural Language Processing (NLP), enabling the system to generate descriptive clinical reports and answer open-ended questions.

2.4.2 Importance and Characteristics in Medicine

Applying generic VLMs to medicine presents unique challenges. General models often hallucinate or refuse to answer medical queries due to safety filters, behaving like a "layperson". Specialized Medical VLMs, such as the one used in our system, address this through:

- **Open-Ended Conversational Capability:** Unlike closed-set classifiers, the model can engage in multi-turn conversations.
- **Domain Specificity:** It is fine-tuned on biomedical literature (e.g., PubMed Central).
- **Data Efficiency:** Utilizing curriculum learning to adapt to medical concepts quickly.

2.4.3 Selected Model: LLaVA-Med

We selected **LLaVA-Med** (Large Language and Vision Assistant for BioMedicine).

Architecture

The architecture follows a streamlined design:

1. **Vision Encoder (BioMedCLIP):** Encodes the medical image X_v into feature vectors Z_v.
2. **Projection Layer (W):** Maps visual features into the language model's embedding space (H_v).
3. **Language Backbone (Vicuna):** Generates coherent clinical responses.

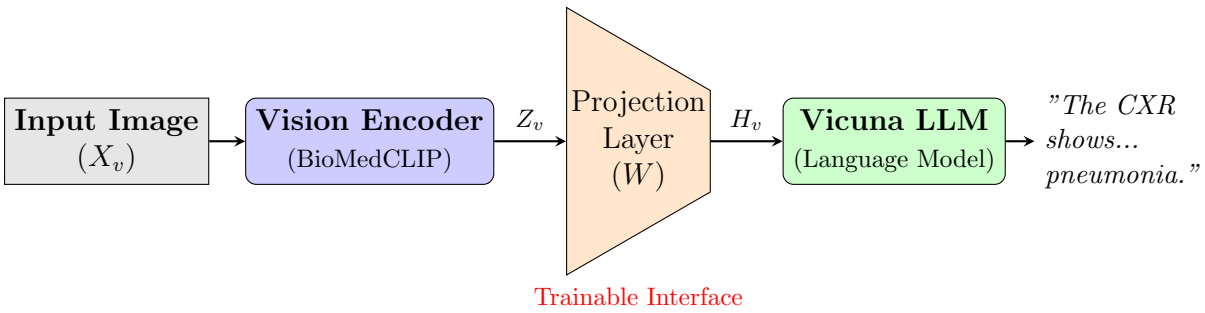

Figure 2.5: LLaVA-Med Architecture: Connecting vision and language via a trainable projection layer.

2.5 Retrieval-Augmented Generation (RAG)

2.5.1 General Overview

Retrieval-Augmented Generation (RAG) is a technique designed to address the inherent limitations of Large Language Models (LLMs), specifically "Hallucination" and "Knowledge Cutoff." **Concept:** Instead of relying solely on the pre-trained parametric memory of the LLM (which acts like a closed book), RAG connects the model to an external, up-to-date knowledge base (open book exam).

The standard RAG process involves three steps:

1. **Retrieval:** The user's query is converted into a vector embedding. The system searches a Vector Database to find document chunks that are semantically similar.

2. **Augmentation:** The retrieved documents are concatenated with the original user query as "Context."

3. **Generation:** The LLM generates an answer based on both the query and the provided context.

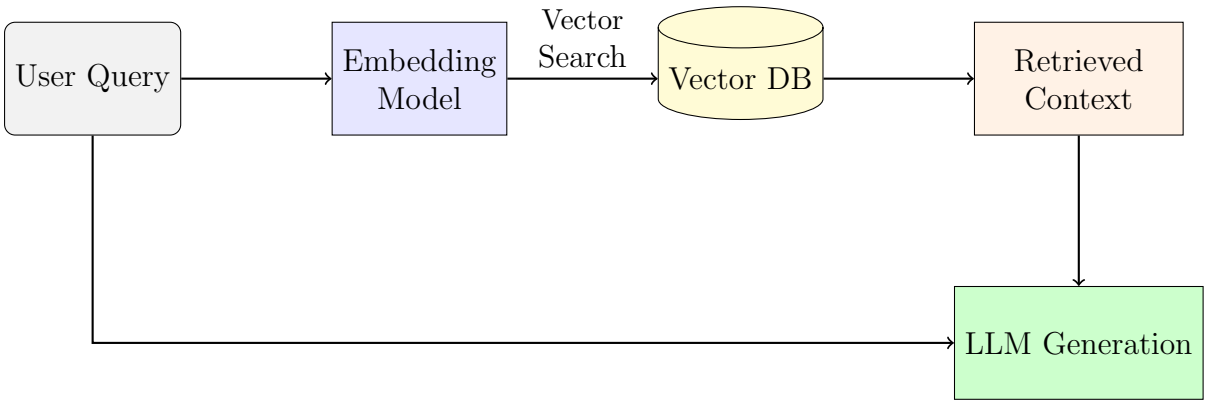

Figure 2.6: Standard RAG Pipeline: Enhancing LLM responses with retrieved context.

2.5.2 Advanced Methodology: Agentic RAG

While standard RAG is powerful, it is a linear pipeline. In complex medical scenarios, this is insufficient. **Asclena** implements **Agentic RAG**, which introduces an autonomous reasoning layer that fuses multi-modal inputs.

Why Agentic?

An "Agent" is an LLM instance equipped with tools and a feedback loop. It does not just answer; it *reasons*.

- **Multi-Modal Fusion:** The agent receives quantitative data (CNN probabilities) AND qualitative descriptions (VLM text).
- **Cross-Verification:** It checks if the text description matches the numerical probability (e.g., if VLM says "Normal" but CNN says "Pneumonia 90%", the Agent flags a conflict).
- **Evidence Retrieval:** It acts as a "Fact-Checker," citing specific sources to resolve ambiguities.

The Verification Workflow in Asclena

The Agentic RAG in our system serves as the **Verification Layer**. It fuses the visual probability matrix (from CNNs) with the semantic description (from LLaVA) and validates them against the stored medical literature.

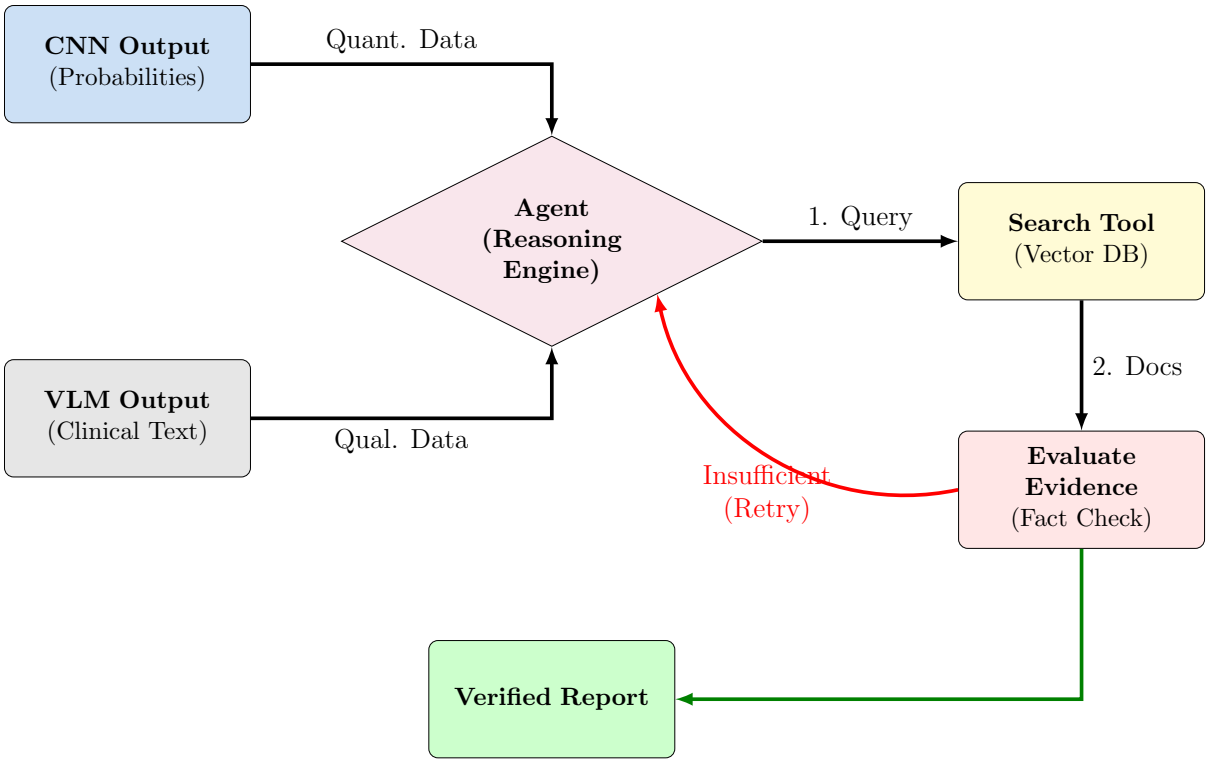

Figure 2.7: Agentic RAG Verification Workflow. The diagram illustrates how the Reasoning Agent fuses quantitative data (CNN) and qualitative descriptions (VLM), verifies them against retrieved medical evidence, and loops back if evidence is insufficient before generating the final report.

2.6 How the System Evaluates and Decides

The Agentic RAG component in Asclena is designed to act as a **verification and adjudication layer** rather than a generator that "trusts" model outputs by default. Even when the Perception model (CNN) and the Interpretation model (VLM) agree, Asclena enforces a strict policy:

> **Evidence-Gating Rule:** No clinical claim is allowed into the final report unless it is supported by retrieved medical evidence.

This rule directly targets the failure modes of medical AI systems, especially overconfident narration, hallucination, and unjustified conclusions. Operationally, the Agentic RAG treats CNN outputs and VLM text as **hypotheses** that must be validated against a curated medical knowledge base.

2.6.1 Inputs to the Agent (Claim Interface)

The agent does not directly "see" the raw scan. Instead, it receives a structured bundle of inputs (a claim interface) produced by the upstream services:

- **Quantitative Evidence (CNN / Perception):** multi-label probabilities $\mathbf{p} = [p_1, \ldots, p_K]$, selected top-$k$ predictions, decision thresholds, and an explainability artifact (e.g., Grad-CAM heatmap path).

- **Qualitative Hypotheses (VLM / Interpretation):** natural-language findings and impressions describing what is observed and what it may imply clinically.
- **Retrieval Tooling (RAG):** a semantic search tool over a medical vector database that returns evidence passages with citations/metadata.

2.6.2 Evaluation Pipeline (Decision Logic)

The evaluation performed by the Agentic RAG can be understood as a deterministic pipeline with a verification loop:

Step 1: Claim Extraction (From Text to Verifiable Units)

The VLM output is first converted into a set of explicit, verifiable claims:

$$\mathcal{C} = \{c_1, c_2, \ldots, c_n\}.$$

Each claim is represented with attributes such as type (finding vs. diagnosis), polarity (present vs. absent), and strength (certain vs. uncertain). Examples include:

- "Opacity is present" (finding, positive).
- "No pleural effusion" (finding, negative).
- "Likely pneumonia" (diagnostic hypothesis, uncertain).

This transformation is essential because verification cannot be performed reliably on free-form text alone.

Step 2: Cross-Model Consistency Check (CNN vs. VLM)

For each claim c_i, the agent checks whether it is:

- **Consistent** with CNN outputs (e.g., high probability for the related label),
- **Contradictory** to CNN outputs (e.g., VLM claims "normal" while CNN indicates strong abnormality),
- **Unverifiable by CNN** (e.g., localization or fine-grained descriptions not covered by available labels).

Importantly, **consistency is never treated as proof**. It is only a confidence signal that influences how conservative the final phrasing should be.

Step 3: Evidence Retrieval (Mandatory Grounding)

For every claim c_i (regardless of agreement), the agent generates one or more retrieval queries to the medical knowledge base. Retrieved passages constitute an evidence set:

$$E(c_i) = \{e_1, e_2, \ldots, e_m\}.$$

Evidence is evaluated by relevance, source quality (e.g., guidelines and peer-reviewed reviews preferred), and direct support for the claim. If evidence is insufficient, the agent revises the query and retries retrieval (verification loop).

Step 4: Evidence Sufficiency and Strength Assessment

After retrieval, the agent assigns each claim an evidence status:

- **Strongly Supported:** evidence directly supports the claim.
- **Weak/Partial Support:** evidence is relevant but not decisive (or only supports a softer phrasing).
- **Unsupported:** evidence is missing or does not support the claim.

Step 5: Final Adjudication (Accept / Accept-with-Uncertainty / Defer)

The agent then decides how (or whether) the claim appears in the final report:

1. **Accept (Evidence-Backed):** if evidence is strong, the claim is included with citations.
2. **Accept with Uncertainty:** if evidence is weak/partial, the claim is included with conservative language (e.g., "may suggest", "possible") and citations.
3. **Defer / Reject:** if evidence is not available, the claim is excluded or explicitly deferred as "not supported by retrieved evidence".

This decision stage enforces the central safety property: **no unsupported clinical statement enters the output**.

2.6.3 Comprehensive Outcome Cases

Given the three information sources (CNN, VLM, Evidence), the most important outcome cases are:

- **Agreement + Strong Evidence:** accept claim with citations and normal confidence.
- **Agreement + Weak Evidence:** accept claim with hedging and explicit uncertainty.
- **Agreement + No Evidence:** defer; do not promote agreement into a clinical statement.
- **Contradiction + Strong Evidence (for one side):** flag inconsistency; prioritize evidence-backed phrasing and reduce confidence.
- **Contradiction + No Evidence:** conservative report; explicitly state that the system cannot verify the claim.
- **CNN-Only Signal (VLM vague) + Evidence:** produce an evidence-grounded summary guided by quantitative confidence.
- **VLM-Only Detail (not label-covered) + Evidence:** include as an observation with cautious wording and citations.

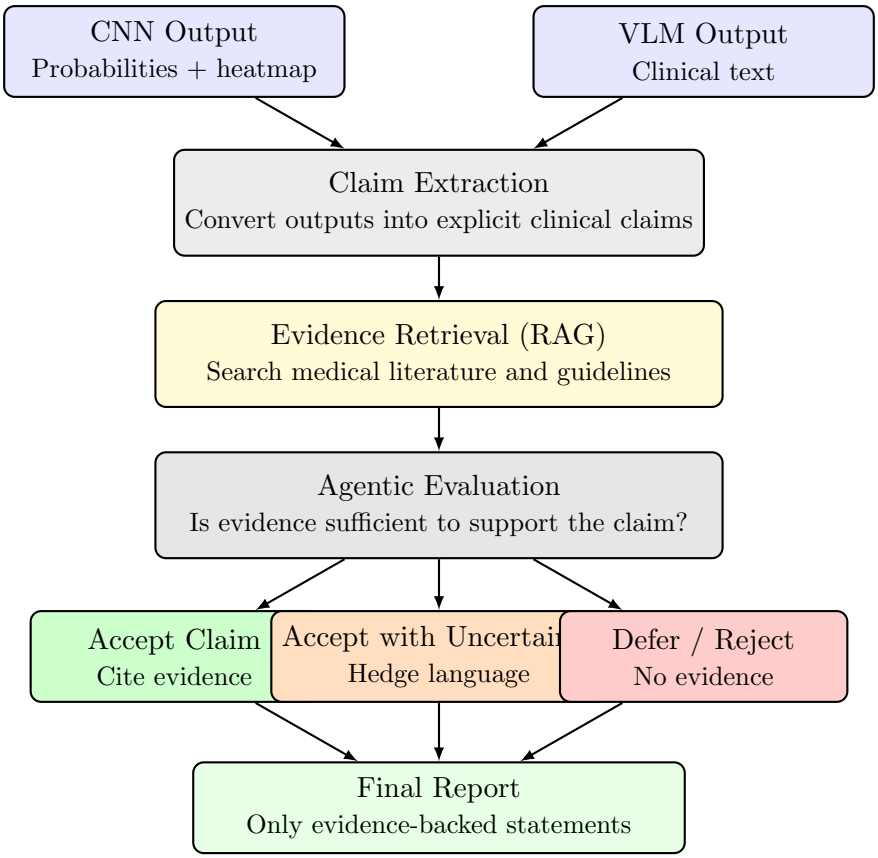

Figure 2.8: Evidence-gated Agentic RAG workflow. All clinical claims generated by CNN and VLM are treated as hypotheses and must be supported by retrieved medical evidence before inclusion in the final report.

2.6.4 Why This Improves Safety and Trust

The primary safety gain arises from transforming model outputs into **auditable, evidence-grounded claims**. Instead of presenting the final answer as a monolithic "AI diagnosis", the system: (i) exposes internal agreement/contradiction, (ii) enforces evidence retrieval for every statement, and (iii) cites sources to justify the report. This shifts the system from "prediction" to **transparent clinical decision support** that is more aligned with real-world medical governance requirements.

Chapter 3

Implementation Architecture & Data Flow

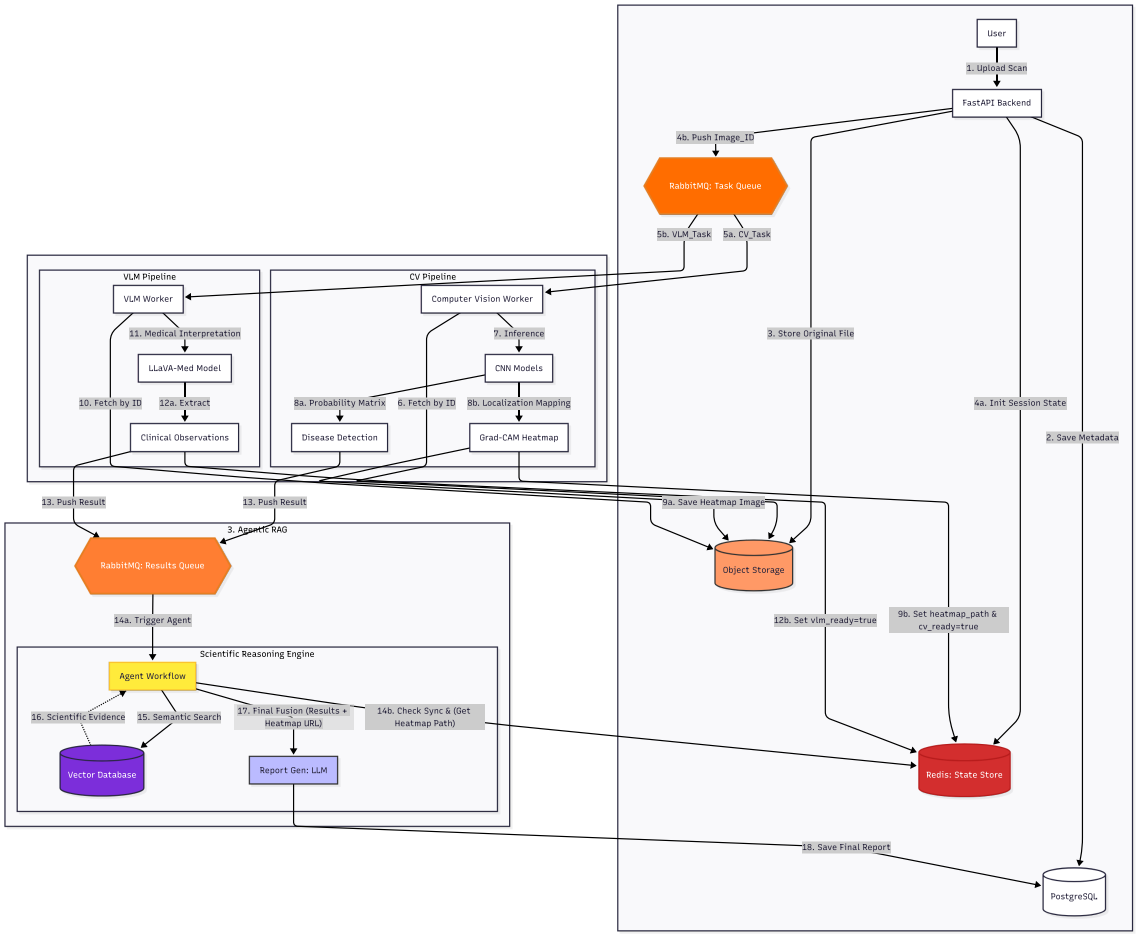

Figure 3.1: High-Level Asclena AI System Architecture.

This chapter details the journey of data within the Asclena AI system, explaining how information flows between microservices, storage layers, and AI models based on the proposed Activity Diagram.

3.1 Data Ingestion Layer

The flow begins when the user uploads a medical scan. To ensure low latency and scalability, the system creates a "Split-Path" for data storage:

- **Binary Data (The Image):** Large files are streamed directly to **Object Storage** (MinIO/S3). This keeps the main database lightweight.
- **Metadata (Context):** Patient ID, timestamps, and file paths are stored in **PostgreSQL** for relational integrity.
- **Task Dispatch:** The API Gateway pushes a lightweight `Task_ID` to **RabbitMQ**, decoupling ingestion from processing.

3.2 Parallel Processing Pipeline

Asclena utilizes a "Fork-Join" pattern to process visual and semantic data simultaneously.

3.2.1 The CV Stream (Perception)

The **Computer Vision Worker** fetches the image and runs the **CNN** model.

- **Output 1:** A probability matrix (e.g., Pneumonia: 0.9).
- **Output 2:** A Class Activation Map (CAM). This heatmap is generated as an image and saved back to Object Storage.
- **State Update:** The worker writes the heatmap URL and probabilities to **Redis** and sets the flag `cv_ready=True`.

3.2.2 The VLM Stream (Interpretation)

The **Vision-Language Worker** runs the **LLaVA-Med** model.

- **Output:** A natural language description of the scan.
- **State Update:** The worker writes the text findings to **Redis** and sets the flag `vlm_ready=True`.

3.3 Synchronization and Fusion (The Join)

Since the CV model (fast) and VLM model (slow) finish at different times, **Redis** acts as the synchronization barrier. The **Agentic RAG** process is triggered only when both `cv_ready` and `vlm_ready` flags are active. This ensures the Agent has a complete picture (Quantitative + Qualitative) before verifying facts against the **Vector Database**.

3.4 Architecture Data Flow Diagram

The following diagram visualizes the exact movement of data packets and control signals throughout the system.

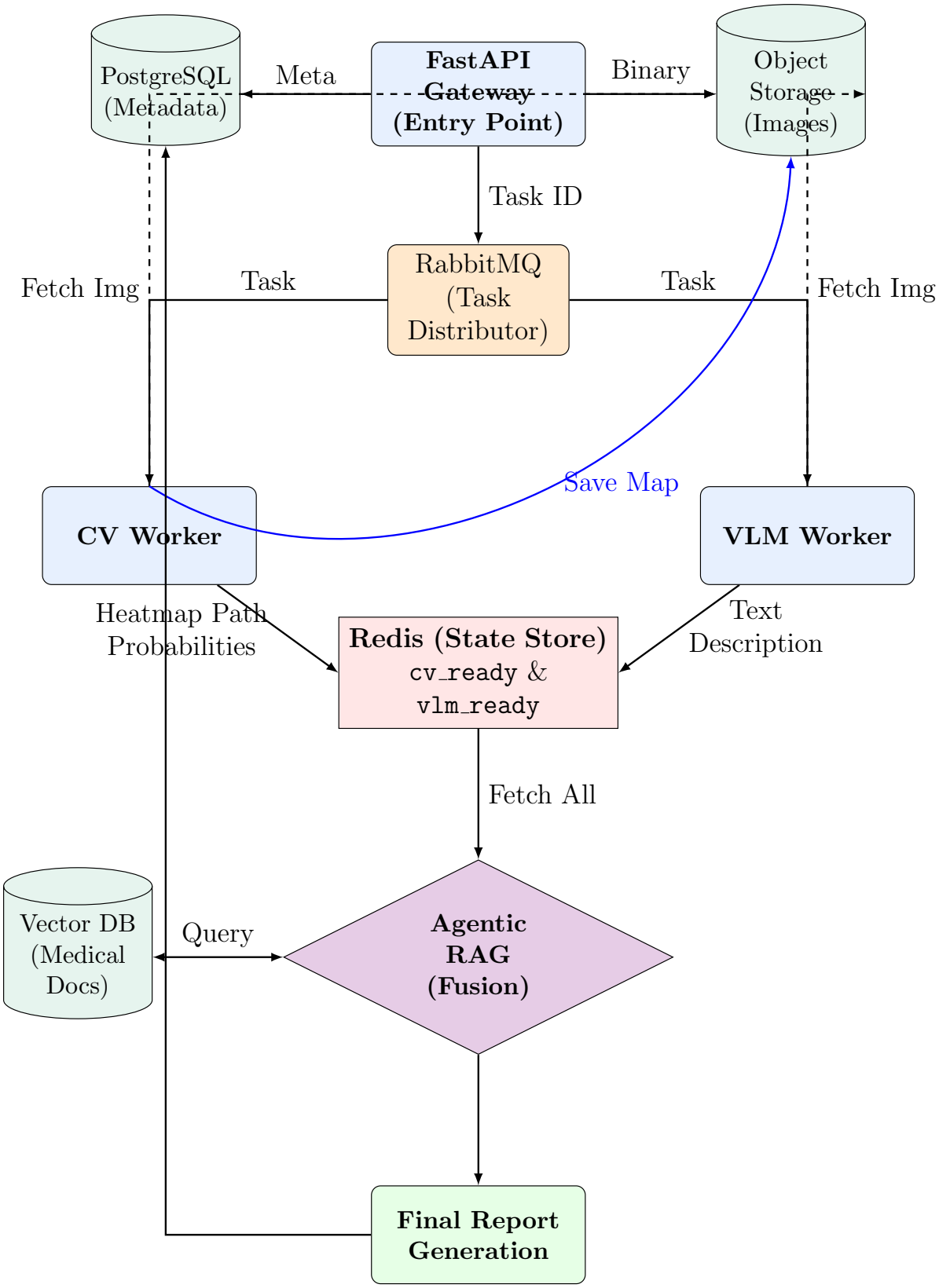

Figure 3.2: Detailed Data Flow Architecture. Shows the separation of binary/metadata storage, the parallel processing fork, Redis synchronization, and the final Agentic fusion.